

BỘ THÔNG TIN VÀ TRUYỀN THÔNG
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO THỰC TẬP THEO TUẦN
NĂM HỌC 2025-2026

TÊN ĐỀ TÀI:
BÀI TOÁN TÓM TẮT VĂN BẢN

Người hướng dẫn: ThS. Phan Thị Hà

Sinh viên thực hiện

Mã sinh viên

Lường Tiến Dũng

B22DCCN128

Hà Nội, 2025

MỤC LỤC

BẢNG TIẾN ĐỘ.....	1
NỘI DUNG THỰC TẬP (TUẦN 1 - 2).....	2
NỘI DUNG THỰC TẬP (TUẦN 3 - 4).....	3
I. Mục tiêu đề ra.....	3
II. Những việc đã làm (Theo đúng kế hoạch đề cương).....	4
III. Mục tiêu dự kiến kỳ báo cáo tiếp theo (Tuần 5 - 6).....	4
DANH MỤC BẢNG.....	5
DANH MỤC HÌNH ẢNH.....	6
CHƯƠNG I: GIỚI THIỆU.....	1
1.1. Mục tiêu và phạm vi đề tài.....	1
1.2. Tổng quan bài toán.....	1
1.3. Nội dung thực hiện trong kỳ báo cáo.....	1
CHƯƠNG II: CƠ SỞ LÝ THUYẾT.....	2
2.1. Bài toán tóm tắt văn bản.....	2
2.1.1. Định nghĩa.....	2
2.1.2. Lý do lựa chọn.....	2
2.1.3. Phân loại bài toán.....	3
2.2. Nền Tảng Lý Thuyết NLP.....	4
2.2.1. Xử lý ngôn ngữ tự nhiên (NLP).....	4
2.2.2. Biểu diễn văn bản (Text Representation).....	5
2.2.3. Tokenization và Word Segmentation cho tiếng Việt.....	7
2.3. Các thể hệ mô hình.....	8
2.3.1. Thế hệ 1 - phương pháp thống kê & đồ thị.....	8
2.3.2. Thế hệ 2: Mạng Nơ-ron Hồi quy (Seq2Seq + Attention).....	13
2.3.3. Thế hệ 3, 4: Kiến trúc Transformer và LLM.....	17
2.4. Phương Pháp Đánh Giá: ROUGE Score.....	18
CHƯƠNG 3: CÀI ĐẶT VÀ ĐÁNH GIÁ THỰC NGHIỆM.....	20
3.1. Tuần 1-2.....	20
3.1.1. Kiến trúc hệ thống sản phẩm.....	20
3.1.2. Tiền xử lý dữ liệu y khoa.....	20
3.1.3. Cài đặt mô hình và Huấn luyện.....	20
3.1.4. Đánh giá kết quả.....	22
3.2. Tuần 3-4.....	23
3.2.1. Thử nghiệm suy luận với mô hình Transformer (ViT5):.....	23

3.2.2. Công tác thu thập dữ liệu (Data Crawling):.....	24
3.3. Kết luận và Định hướng kỳ tiếp theo.....	25
TÀI LIỆU THAM KHẢO.....	26

BẢNG TIẾN ĐỘ

Thời gian	Công việc đã làm	Link github
Tuần 1 - 2 (29/06 - 12/07)	• Tìm hiểu về bài toán text-sumarization • Tìm hiểu về Extractive Summarization(trích xuất) và Abstractive Summarization (trùng tượng) • Tìm hiểu thuật toán text-rank và cài đặt chạy thử nghiệm • Tìm hiểu về PhoBert cài đặt và chạy thử (gọi mô hình) • Cài đặt Sequence-to-Sequence sử dụng mạng LSTM kết hợp cơ chế Attention (Chú ý). • Xây dựng 1 bộ dữ liệu để chạy cho seq2seq vừa xây dựng để train và chạy thử.	link github
Tuần 3-4 (13/07 - 26/07)	• Nghiên cứu kiến trúc Transformer và LLM (ViT5). • Xây dựng mã nguồn đánh giá (Inference Smoke Test) và hàm tính ROUGE score. • Thu thập bộ dữ liệu y khoa thô chuẩn bị cho việc huấn luyện.	

NỘI DUNG THỰC TẬP (TUẦN 1 - 2)

I. Mục tiêu đề ra

Tiếp cận bài toán Tóm tắt văn bản tiếng Việt (Vietnamese Text Summarization) chuyên sâu trong miền dữ liệu Y khoa.

Khảo sát và đánh giá 2 hướng tiếp cận kinh điển:

- **Thế hệ 1:** Tóm tắt trích xuất (Extractive Summarization) với thuật toán TextRank.
- **Thế hệ 2:** Tóm tắt trừu tượng (Abstractive Summarization) với mạng học sâu Seq2Seq LSTM kết hợp Attention.

Xây dựng quy trình khép kín từ khâu tiền xử lý dữ liệu đến huấn luyện và đánh giá tự động (Evaluation).

II. Những việc đã làm

Nghiên cứu lý thuyết:

- Đọc hiểu thuật toán TextRank dùng trong xử lý ngôn ngữ tự nhiên, phương pháp xây dựng đồ thị câu (Sentence Graph) nhằm trích xuất nguyên vẹn các câu cốt lõi.
- Tìm hiểu kiến trúc mạng Sequence-to-Sequence (Encoder-Decoder) bằng mạng nơ-ron hồi quy LSTM.
- Phân tích cơ chế Attention (giúp mô hình liếc nhìn và tập trung vào từ khóa) và vai trò của các token điều hướng chuyên dụng (SOS, EOS, UNK).

Thực hành lập trình (Framework PyTorch):

- Dữ liệu: Tiền xử lý bộ dữ liệu y khoa (gần 7000 bài báo), xây dựng hàm chuẩn hóa cấu hình bắt buộc giữ lại các chữ số (đảm bảo độ chính xác cho thông tin liều lượng, tuổi tác y khoa). Xây dựng từ điển phân tách Train/Test.
- Cài đặt mô hình: Lập trình thành công thuật toán TextRank cơ bản (cho tốc độ rất nhanh nhưng câu văn bị rời rạc). Xây dựng khối Deep Learning (EncoderRNN,

AttnDecoderRNN) có tích hợp kỹ thuật Teacher Forcing và Attention Masking nhằm giúp hệ thống tự sinh ra câu mới.

- Suy luận & Đánh giá: Thực thi mô hình sinh tóm tắt tự động trên tập kiểm thử (Test) và đo lường tự động bằng độ đo ROUGE Score. Mức độ trùng lặp ROUGE-1 dao động ở mức 0.21. Ghi nhận nguyên nhân điểm thấp là do giới hạn cốt lõi của LSTM với văn bản y khoa quá dài (hiện tượng Vanishing Gradient) và việc mạng nơ-ron chưa được huấn luyện kiến thức tiếng Việt từ trước (Pre-train).

III. Mục tiêu dự kiến kỳ báo cáo tiếp theo (Tuần 3 - 4)

Chuyển hướng sang kiến trúc mạng Transformer (Thế hệ 3, 4) để giải quyết triệt để điểm yếu quên ngữ cảnh của LSTM.

Tìm hiểu các Mô hình Ngôn ngữ Lớn (LLM) đã được pre-train sẵn cho tiếng Việt như PhoBERT (tối ưu cho trích xuất) hoặc ViT5 (tối ưu cho trừu tượng).

Nghiên cứu ứng dụng bộ dữ liệu tổng hợp (fine-tune) để huấn luyện mô hình làm quen văn phong tiếng Việt, sau đó tiến hành tinh chỉnh (Fine-tuning) chuyên biệt trên bộ dữ liệu Y khoa nhằm cải thiện đột phá chất lượng tóm tắt và tối ưu ROUGE Score.

NỘI DUNG THỰC TẬP (TUẦN 3 - 4)

I. Mục tiêu đề ra

Chuyển hướng nghiên cứu sang kiến trúc mạng Transformer (Thế hệ 3, 4) và các Mô hình Ngôn ngữ Lớn (LLM) chuyên dụng cho tiếng Việt như ViT5.

Thử nghiệm chạy suy luận (Inference Smoke Test) với mô hình pre-trained để thiết lập đường ống (pipeline) đánh giá ROUGE score tự động.

Tiến hành thu thập và xây dựng bộ dữ liệu Y khoa chuẩn bị cho giai đoạn Fine-tuning sắp tới.

II. Những việc đã làm (Theo đúng kế hoạch đề cương)

Nghiên cứu lý thuyết:

- Tìm hiểu chuyên sâu về kiến trúc Transformer (Self-Attention, Multi-head Attention) giúp giải quyết vấn đề quên ngữ cảnh của LSTM.
- Đọc hiểu về mô hình ViT5 (kiến trúc Encoder-Decoder dựa trên T5) - mô hình Generative LLM tiên tiến nhất hiện nay cho bài toán tóm tắt tiếng Việt.

Thực hành lập trình & Thu thập dữ liệu:

- Xây dựng cấu trúc mã nguồn (Source Code) chuẩn hóa bằng Python: tổ chức các module ``data.py``, ``model.py``, ``metrics.py`` và ``text.py``.
- Chạy thử nghiệm thành công Inference Smoke Test (``test_with_colab.ipynb``) sử dụng mô hình ViT5 gốc từ HuggingFace, kết hợp cài đặt hàm tính ROUGE score tự động.
- Thu thập thành công tập dữ liệu ``vietnews_medical_raw_1000.csv`` bao gồm 1.000 bài báo chuyên ngành y khoa, tạo tiền đề cho quá trình huấn luyện.

III. Mục tiêu dự kiến kỳ báo cáo tiếp theo (Tuần 5 - 6)

Tiền xử lý (Preprocessing) bộ dữ liệu Y khoa vừa thu thập (làm sạch văn bản, áp dụng kỹ thuật Text Chunking).

Tiến hành Fine-tuning mô hình ViT5 trên bộ dữ liệu Y khoa sạch để nâng cao chất lượng tóm tắt chuyên ngành.

Phát triển giao diện Web Demo (Streamlit hoặc FastAPI) để đóng gói sản phẩm hoàn chỉnh.

DANH MỤC BẢNG

Bảng 2.1: các vấn đề để lựa chọn text-sumarization	3
Bảng 2.2: Bảng so sánh extractive vs abstractive	4
Bảng 2.3: Bảng công cụ tách từ	8
Bảng 2.5: Bảng ưu nhược của attention	18

DANH MỤC HÌNH ẢNH

Hình 2.1. Code mẫu tách từ sử dụng	9
Hình 2.2: code mẫu text-rank	14
Bảng 2.4: Bảng ưu nhược của text-	14
Hình 2.3: kiến trúc seq2seq	15
Hình 3.1: quá trình huấn luyện	21
Hình 3.2: Kết quả đánh giá mô	22

CHƯƠNG I: GIỚI THIỆU

1.1. Mục tiêu và phạm vi đề tài

Bài toán Tóm tắt văn bản (Text Summarization) đang là một trong những bài toán cốt lõi của Xử lý ngôn ngữ tự nhiên (NLP). Đề tài này hướng tới việc nghiên cứu, cài đặt và tối ưu hóa các mô hình học sâu (Deep Learning) để tự động hóa việc tóm tắt thông tin. Phạm vi dữ liệu được tập trung vào miền y khoa tiếng Việt, nơi đòi hỏi độ chính xác cao về mặt thuật ngữ, liều lượng và thời gian.

1.2. Tổng quan bài toán

Có hai phương pháp tiếp cận chính để giải quyết bài toán tóm tắt văn bản:

Tóm tắt trích xuất (Extractive Summarization): Tìm kiếm và rút trích các câu quan trọng nhất trong văn bản gốc để ghép thành một đoạn tóm tắt.

Tóm tắt trừu tượng (Abstractive Summarization): Máy tính tự đọc hiểu văn bản gốc và sinh ra các câu từ mới, giống hệt cách biên tập viên con người làm việc.

1.3. Nội dung thực hiện trong kỳ báo cáo

Khảo sát lý thuyết và thuật toán của hai thể hệ mô hình: TextRank (Thể hệ 1) và Seq2Seq LSTM (Thể hệ 2).

Tiến hành cài đặt, xử lý dữ liệu và đánh giá thực nghiệm kiến trúc hệ thống trên tập dữ liệu gồm gần 7000 bài báo y khoa.

CHƯƠNG II: CƠ SỞ LÝ THUYẾT

2.1. Bài toán tóm tắt văn bản

2.1.1. Định nghĩa

Tóm tắt văn bản tự động (Automatic Text Summarization - ATS) là một bài toán cốt lõi trong lĩnh vực **Xử lý Ngôn ngữ Tự nhiên (Natural Language Processing - NLP)**. Mục tiêu của bài toán là tạo ra một phiên bản rút gọn của một hoặc nhiều văn bản nguồn, sao cho bản tóm tắt vẫn giữ lại được những thông tin quan trọng nhất và ý nghĩa cốt lõi của văn bản gốc.

Định nghĩa hình thức:

Cho một văn bản đầu vào D có độ dài n câu, bài toán tóm tắt yêu cầu sinh ra một văn bản đầu ra S có độ dài m câu (với $m \ll n$), sao cho S chứa đựng tối đa lượng thông tin quan trọng từ D .

2.1.2. Lý do lựa chọn

Trong thời đại bùng nổ thông tin số, lượng dữ liệu văn bản được tạo ra mỗi ngày là khổng lồ. Tóm tắt văn bản tự động giúp giải quyết các vấn đề sau:

Vấn đề	Giải pháp của Text Summarization
Quá tải thông tin (Information Overload)	Rút gọn hàng nghìn bài báo, báo cáo thành các bản tóm tắt ngắn gọn, giúp người đọc nhanh chóng nắm bắt nội dung chính
Tiết kiệm thời gian	Thay vì đọc toàn bộ tài liệu dài 50+ trang, người dùng chỉ cần đọc bản tóm tắt 1-2 đoạn

Hỗ trợ ra quyết định	Cung cấp thông tin cốt lõi nhanh chóng cho các nhà quản lý, nhà nghiên cứu
Tăng khả năng tiếp cận	Giúp người khiếm thị hoặc người có hạn chế về thời gian tiếp cận thông tin dễ dàng hơn
Ứng dụng trong hệ thống lớn	Làm đầu vào cho các hệ thống tìm kiếm, chatbot, hệ thống gợi ý

Bảng 2.1: các vấn đề để lựa chọn text-summarization

2.1.3. Phân loại bài toán

Bài toán tóm tắt văn bản được phân loại theo nhiều tiêu chí khác nhau:

A. Theo phương pháp tiếp cận (Quan trọng nhất)

- *Extractive (Trích xuất)*: Chọn lọc các câu quan trọng nhất từ văn bản gốc → Giống như dùng bút highlight đánh dấu
- *Abstractive (Trình tợ / Sinh văn bản)*: Tạo ra các câu mới diễn đạt lại nội dung → Giống như con người viết tóm tắt
- *hybrid (Kết hợp)*: Trích xuất câu quan trọng → Viết lại cho mạch lạc → Kết hợp cả hai phương pháp trên

So sánh chi tiết Extractive vs Abstractive:

Tiêu chí	Extractive (Trích xuất)	Abstractive (Trình tợ)
Cách hoạt động	Chọn và ghép các câu gốc	Sinh ra câu hoàn toàn mới
Ví von	Bút highlight	Người viết tóm tắt 🖋️
Độ trung thực	Rất cao (dùng nguyên câu gốc)	Có rủi ro "ảo giác" (hallucination)

Tính mạch lạc	Thấp hơn (câu có thể rời rạc)	Cao hơn (câu được viết lại liền mạch)
Độ phức tạp kỹ thuật	Đơn giản hơn	Phức tạp hơn nhiều

Bảng 2.2: Bảng so sánh extractive vs abstractive

B. Theo số lượng văn bản đầu vào

Single-Document Summarization (Tóm tắt đơn văn bản): Tóm tắt từ 1 tài liệu duy nhất.

Multi-Document Summarization (Tóm tắt đa văn bản): Tổng hợp thông tin từ nhiều tài liệu liên quan.

C. Theo mục đích

Generic Summarization (Tóm tắt chung): Tóm tắt toàn bộ nội dung chính.

Query-focused Summarization (Tóm tắt theo truy vấn): Tóm tắt theo một câu hỏi hoặc chủ đề cụ thể.

2.2. Nền Tảng Lý Thuyết NLP

2.2.1. Xử lý ngôn ngữ tự nhiên (NLP)

NLP (Natural Language Processing) là một nhánh của trí tuệ nhân tạo (AI) tập trung vào việc giúp máy tính hiểu, diễn giải và sinh ra ngôn ngữ tự nhiên của con người.

Các tầng xử lý ngôn ngữ tự nhiên:

Tầng Ứng dụng (Application Layer)
Tóm tắt, Dịch máy, Chatbot, Phân tích cảm xúc
Tầng Ngữ nghĩa (Semantic Layer)
Hiểu ý nghĩa câu, Quan hệ giữa các thực thể
Tầng Cú pháp (Syntactic Layer)
Phân tích ngữ pháp, Cây cú pháp, POS Tagging
Tầng Từ vựng (Lexical Layer)
Tokenization, Tách từ, Lemmatization
Tầng Tiền xử lý (Preprocessing)
Làm sạch text, Loại bỏ stop words, Chuẩn hóa

2.2.2. Biểu diễn văn bản (Text Representation)

Máy tính không thể trực tiếp hiểu ngôn ngữ tự nhiên. Do đó, văn bản cần được chuyển đổi thành dạng số (vector). Có nhiều phương pháp biểu diễn:

a) Bag of Words (BoW)

Biểu diễn văn bản dưới dạng tần suất xuất hiện của từ, bỏ qua thứ tự.

Câu: "Tôi yêu Việt Nam, Việt Nam tươi đẹp"

Vector: {Tôi: 1, yêu: 1, Việt_Nam: 2, tươi: 1, đẹp: 1}

b) TF-IDF (Term Frequency - Inverse Document Frequency)

Cải tiến BoW bằng cách đánh giá mức độ quan trọng của từ:

TF (Term Frequency): Tần suất xuất hiện của từ trong văn bản.

IDF (Inverse Document Frequency): Mức độ "hiếm" của từ trong toàn bộ tập văn bản.

Công thức:

$$TF(t, d) = (\text{Số lần từ } t \text{ xuất hiện trong văn bản } d) / (\text{Tổng số từ trong } d)$$

$$IDF(t) = \log(N / df(t))$$

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

Trong đó: N = tổng số văn bản, $df(t)$ = số văn bản chứa từ t

Ý nghĩa: Từ xuất hiện nhiều trong 1 văn bản nhưng ít trong các văn bản khác → TF-IDF cao → từ đó quan trọng cho văn bản đó.

c) Word Embeddings (Word2Vec, GloVe, FastText)

Biểu diễn mỗi từ bằng một vector dày đặc (dense vector) trong không gian nhiều chiều, sao cho các từ có nghĩa tương tự nằm gần nhau.

Ví dụ (Word2Vec):

"vua" = [0.2, 0.8, -0.1, 0.5, ...] (vector 300 chiều)

"nữ hoàng" = [0.3, 0.7, 0.2, 0.4, ...]

Tính chất thú vị:

$$\text{vector}(\text{"vua"}) - \text{vector}(\text{"nam"}) + \text{vector}(\text{"nữ"}) \approx \text{vector}(\text{"nữ hoàng"})$$

d) Contextual Embeddings (BERT, PhoBERT)

Khắc phục hạn chế của Word Embeddings truyền thống (mỗi từ chỉ có 1 vector cố định). Contextual Embeddings tạo ra vector khác nhau cho cùng 1 từ tùy vào ngữ cảnh:

Câu 1: "Tôi đến ngân hàng rút tiền" → "ngân hàng" = [0.9, 0.1, ...] (tài chính)

Câu 2: "Ngồi trên bờ ngân hàng sông" → "ngân hàng" = [0.1, 0.8, ...] (địa lý)

2.2.3. Tokenization và Word Segmentation cho tiếng Việt

Tiếng Việt là ngôn ngữ đơn lập (isolating language) với đặc điểm:

Từ ghép (compound words): Nhiều từ trong tiếng Việt được tạo thành từ 2+ tiếng (âm tiết). Ví dụ: "học_sinh", "trường_đại_học", "xử_lý_ngôn_ngữ".

Không có dấu phân cách từ rõ ràng: Không giống tiếng Anh (mỗi từ cách nhau bởi dấu cách), trong tiếng Việt, dấu cách chỉ phân tách tiếng, không phải từ.

Ví dụ:

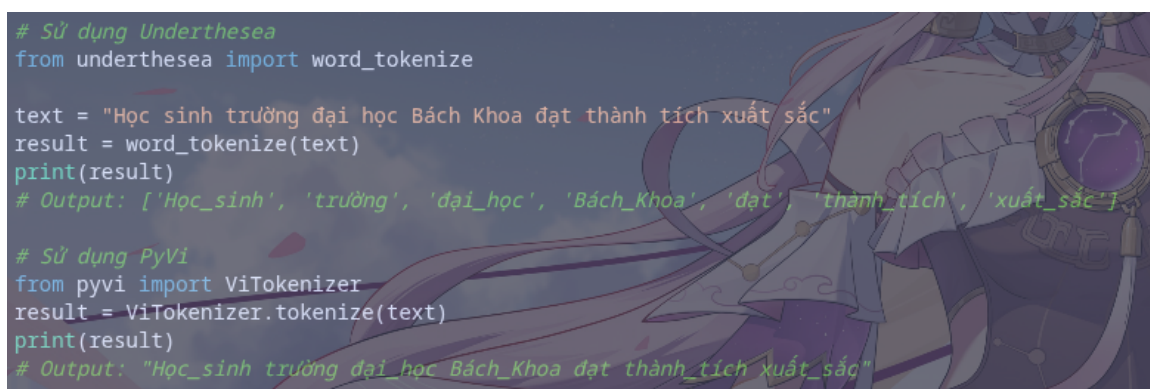
Input: "Học sinh trường đại học Bách Khoa"

Tách tiếng (syllable): ["Học", "sinh", "trường", "đại", "học", "Bách", "Khoa"]

Tách từ (word): ["Học_sinh", "trường", "đại_học", "Bách_Khoa"]

Công cụ	Mô tả	Cài đặt
VnCoreNLP (RDRSegmenter)	Công cụ mạnh nhất, được dùng trong PhoBERT	<code>pip install py_vncorenlp</code>
Underthesea	Thư viện NLP tiếng Việt toàn diện	<code>pip install underthesea</code>
PyVi	Thư viện nhẹ, dễ sử dụng	<code>pip install pyvi</code>

Bảng 2.3: Bảng công cụ tách từ



```

# Sử dụng Underthesea
from underthesea import word_tokenize

text = "Học sinh trường đại học Bách Khoa đạt thành tích xuất sắc"
result = word_tokenize(text)
print(result)
# Output: ['Học_sinh', 'trường', 'đại_học', 'Bách_Khoa', 'đạt', 'thành_tích', 'xuất_sắc']

# Sử dụng PyVi
from pyvi import ViTokenizer
result = ViTokenizer.tokenize(text)
print(result)
# Output: "Học_sinh trường đại_học Bách_Khoa đạt thành_tích xuất_sắc"

```

Hình 2.1. Code mẫu tách từ sử dụng

2.3. Các thể hệ mô hình

2.3.1. Thế hệ 1 - phương pháp thống kê & đồ thị

2.3.1.1. Tổng quan

Đây là phương pháp **không giám sát (unsupervised)**, không cần dữ liệu huấn luyện. Coi văn bản như một đồ thị (graph), trong đó mỗi câu là một đỉnh (node), và cạnh (edge) giữa các đỉnh thể hiện mức độ tương tự giữa các câu.

2.3.1.2. TextRank — Thuật toán chi tiết

Nguồn gốc: Được đề xuất bởi Mihalcea và Tarau (2004), lấy cảm hứng từ thuật toán PageRank của Google.

Nguyên lý hoạt động:

PageRank xếp hạng trang web dựa trên số lượng và chất lượng liên kết đến nó. TextRank áp dụng ý tưởng tương tự: **một câu quan trọng nếu nó có liên kết (tương tự) với nhiều câu quan trọng khác.**

Các bước thực hiện

Bước 1: Tiền xử lý văn bản

- Làm sạch văn bản, chuẩn hóa chữ thường, loại bỏ ký tự đặc biệt, stop words...

Bước 2: Tách văn bản thành các câu riêng lẻ

- Sử dụng các công cụ tách câu dựa trên dấu chấm, dấu hỏi hoặc các thư viện NLP.

Bước 3: Xây dựng đồ thị

- Mỗi câu đóng vai trò là một nút (Node) trên đồ thị.
- Tính độ tương tự giữa mọi cặp câu bằng phương pháp Cosine Similarity.
- Tạo các cạnh có trọng số (Weighted Edges) giữa các câu.

Bước 4: Chạy thuật toán xếp hạng (PageRank)

- Lặp đi lặp lại quá trình cập nhật cho đến khi điểm số hội tụ.

Bước 5: Chọn N câu có điểm cao nhất làm bản tóm tắt

- Sắp xếp theo thứ tự giảm dần và chọn ra các câu đứng đầu.

Công thức tính điểm TextRank

$$WS(V_i) = (1 - d) + d \times \sum [(w_{ij} / \sum w_{ij}) \times WS(V_j)]$$

- **WS(V_i):** Điểm quan trọng của câu \$i\$.
- **d:** Hệ số giảm chấn (damping factor), thường được thiết lập mặc định bằng 0.85.
- **w_{ij}:** Trọng số cạnh giữa câu \$i\$ và câu \$j\$ (độ tương tự giữa 2 câu).
- **V_j:** Các câu có liên kết với câu \$i\$.

Cách tính độ tương tự giữa 2 câu (Cosine Similarity)

$$\text{Cosine}(A, B) = (A \cdot B) / (\|A\| \times \|B\|)$$

- **A, B:** Vector biểu diễn TF-IDF hoặc Bag-of-Words (BoW) của 2 câu.
- **A · B:** Tích vô hướng (dot product) giữa hai vector.
- **||A||, ||B||:** Độ dài (norm) hình học của từng vector tương ứng.

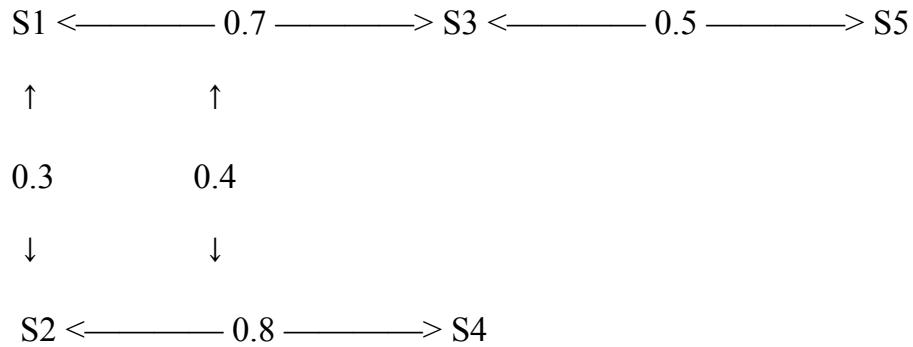
Ví dụ minh họa: Giả sử văn bản gốc gồm 5 câu: **S1, S2, S3, S4, S5.**

Mã trận tương tự:

	S1	S2	S3	S4	S5
S1	1.0	0.3	0.7	0.1	0.2
S2	0.3	1.0	0.4	0.8	0.1
S3	0.7	0.4	1.0	0.2	0.5
S4	0.1	0.8	0.2	1.0	0.3
S5	0.2	0.1	0.5	0.3	1.0

Sơ đồ cấu trúc liên kết đồ thị:

Plaintext



Kết quả xếp hạng (PageRank) & Tóm tắt

Sau khi chạy thuật toán lặp cho đến khi điểm số hội tụ:

S3: 0.28 ← Điểm cao nhất (do liên kết mạnh với cả S1 và S5)

S2: 0.24

S1: 0.20

S4: 0.16

S5: 0.12

→ **Kết luận:** Chọn các câu **S3, S2, S1** làm bản tóm tắt (Top-3 câu có điểm cao nhất).

```

import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import networkx as nx

def textrank_summarize(text, num_sentences=3):
    """
    Tóm tắt văn bản sử dụng thuật toán TextRank.

    Args:
        text (str): Văn bản cần tóm tắt
        num_sentences (int): Số câu trong bản tóm tắt

    Returns:
        str: Bản tóm tắt
    """
    # Bước 1: Tách câu
    sentences = text.split('.')
    sentences = [s.strip() for s in sentences if len(s.strip()) > 10]

    if len(sentences) <= num_sentences:
        return text

    # Bước 2: Tạo vector TF-IDF cho mỗi câu
    vectorizer = TfidfVectorizer()
    tfidf_matrix = vectorizer.fit_transform(sentences)

    # Bước 3: Tính ma trận tương tự (Cosine Similarity)
    similarity_matrix = cosine_similarity(tfidf_matrix)

    # Bước 4: Xây dựng đồ thị và chạy PageRank
    graph = nx.from_numpy_array(similarity_matrix)
    scores = nx.pagerank(graph, alpha=0.85) # alpha = damping factor

    # Bước 5: Xếp hạng câu và chọn top-N
    ranked_sentences = sorted(
        ((scores[i], i, s) for i, s in enumerate(sentences)),
        reverse=True
    )

    # Sắp xếp lại theo thứ tự xuất hiện trong văn bản gốc
    selected = sorted(ranked_sentences[:num_sentences], key=lambda x: x[1])
    summary = '. '.join([s for _, _, s in selected]) + '.'

    return summary

# Sử dụng
text = """
Trí tuệ nhân tạo đang thay đổi cách chúng ta sống và làm việc.
Các ứng dụng AI đã xuất hiện trong nhiều lĩnh vực từ y tế đến giáo dục.
Đặc biệt trong lĩnh vực xử lý ngôn ngữ tự nhiên, các mô hình ngôn ngữ lớn
đã đạt được những bước tiến đáng kinh ngạc. Tuy nhiên, việc ứng dụng AI
cũng đặt ra nhiều thách thức về đạo đức và quyền riêng tư.
Các chính phủ trên thế giới đang nỗ lực xây dựng khung pháp lý cho AI.
"""

print(textrank_summarize(text, num_sentences=2))

```

Hình 2.2: code mẫu text-rank

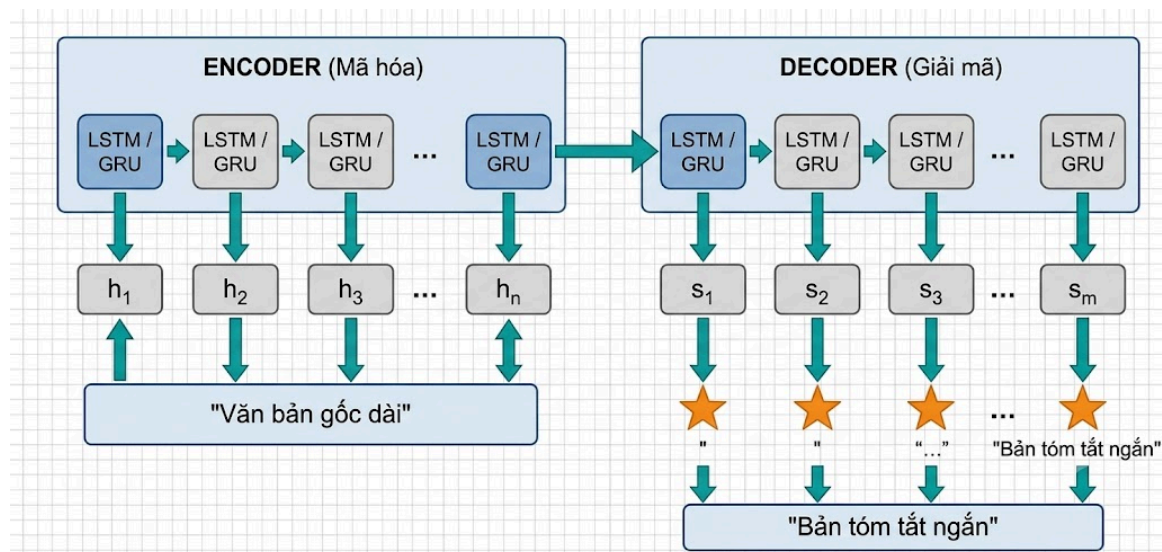
Ưu điểm	Hạn chế
Không cần dữ liệu huấn luyện	Chỉ làm Extractive (không thể diễn đạt lại)
Triển khai nhanh, nhẹ	Phụ thuộc nhiều vào cấu trúc câu gốc
Hoạt động với bất kỳ ngôn ngữ nào	Kết quả có thể thiếu mạch lạc
Độ trung thực cao (dùng câu gốc)	Không hiểu ngữ nghĩa sâu

Bảng 2.4: Bảng ưu nhược của text-

2.3.2. Thể hệ 2: Mạng Nơ-ron Hồi quy (Seq2Seq + Attention)

Đây là thể hệ mô hình đầu tiên có khả năng sinh ra câu mới (Abstractive), đánh dấu bước chuyển mình quan trọng từ 'trích xuất' sang 'tạo mới'. Kiến trúc cốt lõi là Sequence-to-Sequence (Seq2Seq) kết hợp với cơ chế Attention.

2.3.2.1. Kiến trúc Sequence-to-Sequence (Seq2Seq)



Hình 2.3: kiến trúc seq2seq

Quy trình hoạt động

1. Encoder đọc lần lượt từng từ của văn bản gốc.

2. Sau mỗi từ, Encoder tạo ra một Hidden State: $h_1, h_2, h_3, \dots, h_N$
3. Hidden State cuối cùng (h_N) được nén thành một Context Vector, chứa thông tin tổng quát của toàn bộ văn bản.
4. Decoder nhận Context Vector làm đầu vào.
5. Decoder sinh từng từ của bản tóm tắt: $s_1 \rightarrow s_2 \rightarrow s_3 \rightarrow \dots \rightarrow s_M$
6. Khi sinh ra ký hiệu kết thúc ($\langle \text{EOS} \rangle$), quá trình tóm tắt kết thúc.

2.3.2.2. Các thành phần chi tiết:

Encoder (Bộ mã hóa)

- Sử dụng mạng LSTM (Long Short-Term Memory) hoặc GRU (Gated Recurrent Unit).
- Đọc văn bản đầu vào tuần tự, từ trái sang phải.
- Tại mỗi bước thời gian t , LSTM nhận từ hiện tại x_t và trạng thái ẩn trước đó h_{t-1} , tạo ra trạng thái ẩn mới h_t .

Decoder (Bộ giải mã)

- Cũng sử dụng LSTM/GRU.
- Khởi tạo bằng context vector từ Encoder.
- Tại mỗi bước, sinh ra 1 từ và dùng từ đó làm đầu vào cho bước tiếp theo

2.3.2.3. Vấn đề của RNN và Mạng LSTM

Mạng nơ-ron hồi quy (RNN) thông thường gặp hiện tượng 'Triệt tiêu đạo hàm' (Vanishing Gradient) khiến mô hình học trước quên sau. Để khắc phục, mạng LSTM (Long Short-Term Memory) được sử dụng.

Cấu trúc lõi của LSTM bao gồm một 'Tế bào nhớ' (Cell State) cùng 3 cổng (Gates) kiểm soát luồng thông tin:

- Cổng quên (Forget Gate): Quyết định thông tin cũ nào nên được loại bỏ khỏi Cell State.
- Cổng nhập (Input Gate): Quyết định thông tin mới nào sẽ được bổ sung vào Cell State.
- Cổng xuất (Output Gate): Quyết định thông tin gì sẽ được xuất ra thành Hidden State tại bước hiện tại.

2.3.2.4. Cơ chế Chú ý (Attention Mechanism)

Vấn đề của Seq2Seq cơ bản: vấn đề nút cổ

Toàn bộ thông tin của văn bản dài bị nén vào một vector duy nhất (context vector). Với văn bản dài (hàng trăm từ), vector này không đủ khả năng lưu giữ hết thông tin → mất thông tin nghiêm trọng.

Giải pháp — Attention Mechanism:

Thay vì chỉ dùng 1 context vector cố định, Attention cho phép Decoder nhìn lại toàn bộ encoder hidden states ở mỗi bước sinh từ.

Encoder hidden states: $[h_1, h_2, h_3, h_4, h_5]$ (5 câu input)

Khi Decoder sinh từ thứ 1:

Attention scores: $[0.1, 0.1, 0.6, 0.1, 0.1]$

↑

Tập trung vào h_3 !

Context vector = $0.1 \times h_1 + 0.1 \times h_2 + 0.6 \times h_3 + 0.1 \times h_4 + 0.1 \times h_5$

= Vector thiên về thông tin của câu 3

Khi Decoder sinh từ thứ 2:

Attention scores: [0.5, 0.2, 0.1, 0.1, 0.1]

↑

Tập trung vào h1!

Context vector = $0.5 \times h_1 + 0.2 \times h_2 + 0.1 \times h_3 + 0.1 \times h_4 + 0.1 \times h_5$

= Vector thiên về thông tin của câu 1

Công thức Attention (Bahdanau Attention):

1. Tính điểm alignment:

$$e_{ij} = \text{score}(s_j, h_i)$$

Trong đó: s_j = trạng thái decoder bước j, h_i = hidden state encoder vị trí i

2. Chuẩn hóa bằng Softmax:

$$\alpha_{ij} = \exp(e_{ij}) / \sum \exp(e_{kj}) \rightarrow \alpha_{ij} = \text{trọng số attention (tổng} = 1)$$

3. Tính context vector:

$$c_j = \sum \alpha_{ij} \times h_i \rightarrow \text{Vector ngữ cảnh động tại bước j}$$

4. Sinh từ tiếp theo:

$$y_j = f(s_j, c_j, y_{j-1})$$

Ưu điểm	Hạn chế
Có thể sinh câu mới (Abstractive)	Xử lý tuần tự $\rightarrow$ chậm, không song song hóa được
Attention giúp xử lý văn bản dài hơn	Vẫn gặp khó khăn với văn bản rất dài (>500 từ)

Hiểu được ngữ cảnh đến một mức nhất định	Vấn đề Vanishing/Exploding Gradient
	Cần lượng dữ liệu huấn luyện lớn

Bảng 2.5: Bảng ưu nhược của attention

2.3.3. Thế hệ 3, 4: Kiến trúc Transformer và LLM

Kiến trúc Transformer ra đời năm 2017 (qua bài báo nổi tiếng 'Attention Is All You Need') đã tạo ra một cuộc cách mạng trong xử lý ngôn ngữ tự nhiên. Điểm đột phá lớn nhất của Transformer là loại bỏ hoàn toàn cơ chế xử lý tuần tự (từng từ một) của mạng nơ-ron hồi quy RNN/LSTM. Thay vào đó, nó tính toán tất cả các từ trong câu cùng một lúc (xử lý song song).

Tại sao Transformer lại vượt trội hơn LSTM?

- Cơ chế Tự chú ý (Self-Attention): Đối với mỗi từ đang xét, mô hình sẽ 'liếc nhìn' toàn bộ các từ khác trong câu để chấm điểm mức độ liên quan. Cơ chế này giống như khi chúng ta đọc một câu dài, não bộ sẽ tự động kết nối các đại từ (như 'nó', 'họ') với danh từ đã nhắc đến ở đầu câu. Nhờ vậy, Transformer không bao giờ bị 'quên' ngữ cảnh dù văn bản y khoa có dài đến đâu.

- Cơ chế Chú ý Đa đầu (Multi-head Attention): Thay vì chỉ phân tích câu ở một khía cạnh, mô hình phân tách ra nhiều 'cái đầu' để học nhiều khía cạnh khác nhau đồng thời (một đầu học ngữ pháp, một đầu học từ vựng chuyên ngành,...).

Sự phát triển thành Mô hình Ngôn ngữ Lớn (LLMs):

Từ khung sườn Transformer, các nhà nghiên cứu xây dựng nên các mô hình khổng lồ, được huấn luyện trước (pre-train) bằng cách đọc hàng tỷ trang tài liệu trên Internet. Chúng được chia làm 3 dạng chính:

- Dạng chỉ dùng Encoder (như BERT, PhoBERT): Chuyên dùng để 'đọc hiểu', phân loại văn bản, trích xuất từ khóa.

- Dạng chỉ dùng Decoder (như GPT, LLaMA): Chuyên dùng để 'viết', sinh ra văn bản tự do.

- Dạng kết hợp Encoder-Decoder (như T5, mBART, ViT5): Đây là cấu trúc hoàn hảo nhất cho bài toán tóm tắt văn bản. Đầu tiên, phần Encoder 'đọc hiểu' toàn bộ bài báo dài, nén thông tin lại. Sau đó, phần Decoder dựa vào thông tin nén đó để 'viết' ra một đoạn tóm tắt ngắn gọn.

Mô hình ViT5 được ứng dụng:

Trong đề tài này, mô hình ViT5 (Vietnamese Text-to-Text Transfer Transformer) do nhóm VietAI phát triển được lựa chọn. ViT5 là mô hình tiên tiến nhất hiện nay dành riêng cho tiếng Việt. Vì đã được 'dạy' tiếng Việt qua hàng trăm GB dữ liệu từ trước (Pre-train), ViT5 hiểu rất rõ ngữ pháp, văn phong và từ đồng nghĩa. Do đó, bản tóm tắt sinh ra sẽ vô cùng trôi chảy, tự nhiên và hiếm khi mắc lỗi diễn đạt.

2.4. Phương Pháp Đánh Giá: ROUGE Score

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) là bộ thang đo tiêu chuẩn để đánh giá chất lượng tóm tắt văn bản. Nguyên lý cốt lõi là so sánh bản tóm tắt do máy sinh ra (Candidate) với bản tóm tắt chuẩn của con người (Reference) thông qua 3 phép đo:

- **ROUGE-1** (Unigram Overlap): Đo lường sự trùng lặp ở cấp độ từ đơn (unigram) giữa bản tóm tắt của mô hình và bản gốc của người viết. Điểm ROUGE-1 cao chứng tỏ mô hình đã giữ lại được hầu hết các từ khóa mang thông tin cốt lõi (ví dụ: tên bệnh, tên thuốc, liều lượng).

- **ROUGE-2** (Bigram Overlap): Đo lường sự trùng lặp ở cấp độ cặp từ liên kề (bigram). Chỉ số này rất quan trọng vì nó đánh giá khả năng giữ nguyên cấu trúc cụm từ, giúp bản tóm tắt không bị xáo trộn trật tự từ và đảm bảo đúng ngữ pháp.

- **ROUGE-L** (Longest Common Subsequence - LCS): Tìm kiếm chuỗi từ chung dài nhất xuất hiện theo đúng thứ tự (không nhất thiết phải đứng sát nhau). Điểm ROUGE-L phản ánh cấu trúc câu và tính mạch lạc của toàn bộ đoạn văn. Trong tóm tắt văn bản, ROUGE-L là chỉ số khắt khe nhất và cũng là thước đo phản ánh gần nhất trải nghiệm đọc của con người.

Mỗi chỉ số ROUGE đều được tính toán theo 3 thành phần:

- **Recall (Độ phủ):** Tỷ lệ số từ/cụm từ khớp nhau chia cho tổng số từ/cụm từ trong bản Reference.

- **Precision (Độ chính xác):** Tỷ lệ số từ/cụm từ khớp nhau chia cho tổng số từ/cụm từ trong bản Candidate.

- **F1-Score:** Trung bình điều hòa giữa Recall và Precision, là con số cuối cùng dùng để xếp hạng mô hình.

CHƯƠNG 3: CÀI ĐẶT VÀ ĐÁNH GIÁ THỰC NGHIỆM

3.1. Tuần 1-2

3.1.1. Kiến trúc hệ thống sản phẩm

Hệ thống được thiết kế theo quy trình (Pipeline) khép kín gồm 4 module cốt lõi:

Khởi Tiên xử lý dữ liệu: Chuẩn hóa, làm sạch văn bản, xây dựng bộ từ điển tự động.

Khối Mã hóa (Encoder): Nén chuỗi đầu vào thành vector trạng thái.

Khối Giải mã (Decoder) & Attention: Sinh từ mới dựa vào trạng thái ẩn và điểm chú ý.

Khối Suy luận & Đánh giá: Chạy test trên dữ liệu mới và đo đạc ROUGE Score.

3.1.2. Tiền xử lý dữ liệu y khoa

Dữ liệu được sử dụng là tập 7000 bài báo y khoa tiếng Việt. Trong bước Tokenization (tách từ) và chuẩn hóa, một thuật toán đặc thù đã được thiết lập để loại bỏ các ký tự đặc biệt gây nhiễu nhưng vẫn giữ nguyên vẹn các chữ số, nhằm bảo toàn tính chính xác sống còn của dữ liệu y khoa (như mốc thời gian, liều lượng, số liệu bệnh án).

Từ điển học máy được khởi tạo với các token điều hướng chuyên dụng: SOS (Bắt đầu câu), EOS (Kết thúc câu) và UNK (Từ lạ).

3.1.3. Cài đặt mô hình và Huấn luyện

Hệ thống được lập trình bằng Python thông qua framework PyTorch. Mô hình TextRank được triển khai để thử nghiệm phương pháp trích xuất (tốc độ cao nhưng câu văn thiếu liên kết).

Sau đó, kiến trúc học sâu được cài đặt với EncoderRNN và AttnDecoderRNN. Trong vòng lặp huấn luyện, kỹ thuật Teacher Forcing (ép mô hình học theo đáp án chuẩn với tỷ lệ 50%) được sử dụng để đẩy nhanh tốc độ hội tụ. Đồng thời, kỹ thuật Attention Masking được áp dụng để loại bỏ nhiễu của các vector padding rỗng.

... Bắt đầu huấn luyện với 3,182 bài báo, 5 epochs...

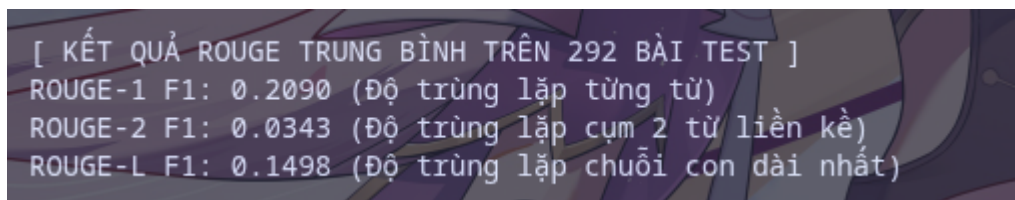
Epoch 1	Bài 100/3,182	Loss TB: 7.8034
Epoch 1	Bài 200/3,182	Loss TB: 7.1911
Epoch 1	Bài 300/3,182	Loss TB: 6.7882
Epoch 1	Bài 400/3,182	Loss TB: 6.7568
Epoch 1	Bài 500/3,182	Loss TB: 6.5687
Epoch 1	Bài 600/3,182	Loss TB: 6.5289
Epoch 1	Bài 700/3,182	Loss TB: 6.4108
Epoch 1	Bài 800/3,182	Loss TB: 6.3992
Epoch 1	Bài 900/3,182	Loss TB: 6.4416
Epoch 1	Bài 1,000/3,182	Loss TB: 6.3591
Epoch 1	Bài 1,100/3,182	Loss TB: 6.2713
Epoch 1	Bài 1,200/3,182	Loss TB: 6.4275
Epoch 1	Bài 1,300/3,182	Loss TB: 6.3120
Epoch 1	Bài 1,400/3,182	Loss TB: 6.3014
Epoch 1	Bài 1,500/3,182	Loss TB: 6.2004
Epoch 1	Bài 1,600/3,182	Loss TB: 6.3067
Epoch 1	Bài 1,700/3,182	Loss TB: 6.2218
Epoch 1	Bài 1,800/3,182	Loss TB: 6.2756
Epoch 1	Bài 1,900/3,182	Loss TB: 6.3005
Epoch 1	Bài 2,000/3,182	Loss TB: 6.1975
Epoch 1	Bài 2,100/3,182	Loss TB: 6.2225
Epoch 1	Bài 2,200/3,182	Loss TB: 6.2609
Epoch 1	Bài 2,300/3,182	Loss TB: 6.1408
Epoch 1	Bài 2,400/3,182	Loss TB: 6.0686
...		
Epoch 5	Bài 2,900/3,182	Loss TB: 4.7861
Epoch 5	Bài 3,000/3,182	Loss TB: 4.8792
Epoch 5	Bài 3,100/3,182	Loss TB: 4.8366

Hình 3.1: quá trình huấn luyện

Tuy bộ data train là có 7000 bài viết nhưng 7000 bài chỉ dùng để cho mô hình lấy các từ và tạo ra được 7590 từ sau đó chỉ huấn luyện với các bài viết > 500 token. Con số 3182 là khá ít cho dữ liệu huấn luyện bởi vì hiện tại chỉ là đang cài đặt chạy thử để hiểu cách seq2seq + attention hoạt động. Và việc hàm loss giảm đã chứng tỏ kết quả huấn luyện không tệ (chưa thể kết luận là huấn luyện tốt).

3.1.4. Đánh giá kết quả

Mô hình tiến hành suy luận (Inference) trên 292 bài báo thuộc tập kiểm thử (Test). Kết quả đánh giá bằng thư viện rouge-score cho thấy ROUGE-1 dao động ở mức 0.21.



```
[ KẾT QUẢ ROUGE TRUNG BÌNH TRÊN 292 BÀI TEST ]  
ROUGE-1 F1: 0.2090 (Độ trùng lặp từng từ)  
ROUGE-2 F1: 0.0343 (Độ trùng lặp cụm 2 từ liền kề)  
ROUGE-L F1: 0.1498 (Độ trùng lặp chuỗi con dài nhất)
```

Hình 3.2: Kết quả đánh giá mô

Nhận xét: Điểm số này là hoàn toàn hợp lý đối với một mạng LSTM được huấn luyện từ đầu (train from scratch) trên tập dữ liệu nhỏ. Mạng LSTM bộc lộ điểm yếu khi gặp văn bản y khoa quá dài và thiếu hụt trầm trọng nền tảng ngữ pháp tiếng Việt do chưa được Pre-train.

3.2. Tuần 3-4

3.2.1. Thử nghiệm suy luận với mô hình Transformer (ViT5):

Để chuẩn bị cho việc áp dụng mô hình ngôn ngữ lớn (LLM) vào bài toán, trong Tuần 3-4, sinh viên đã tiến hành nghiên cứu kiến trúc Transformer và lựa chọn mô hình ViT5 (Vietnamese Text-to-Text Transfer Transformer). Nhằm xác minh khả năng của mô hình trước khi huấn luyện sâu, một quy trình thử nghiệm (Inference Smoke Test) đã được thiết lập thành công trên môi trường Google Colab.

Toàn bộ mã nguồn được cấu trúc hóa module chuyên nghiệp trong thư mục `src/`, bao gồm các file xử lý dữ liệu (`data.py`), kiến trúc mô hình (`model.py`), và hàm đánh giá ROUGE score tự động (`metrics.py`).

3.2.1.1. Kết quả kiểm thử ROUGE Score bước đầu (Inference Smoke Test):

Dựa trên 5 mẫu thử nghiệm ngẫu nhiên, mặc dù chưa được Fine-tuning chuyên biệt trên dữ liệu Y khoa, mô hình ViT5 (phiên bản pre-trained gốc) đã cho kết quả đo lường cực kỳ ấn tượng ngay trong bài test nhanh:

- ROUGE-1 (F1-Score): Đạt 66.01% (Tỷ lệ bao phủ các từ vựng cốt lõi rất cao).
- ROUGE-2 (F1-Score): Đạt 36.11% (Khả năng giữ nguyên các cụm từ chuyên môn đi liền kề nhau khá tốt).
- ROUGE-L (F1-Score): Đạt 41.58% (Cấu trúc câu văn sinh ra rất mạch lạc, liền mạch).
- Hiệu suất nén (Compression): ~92.28% (Bài báo gốc trung bình dài ~437 từ được nén sắc bén xuống thành đoạn tóm tắt trung bình chỉ 33 từ).

3.2.1.2. Giải thích kết quả:

- Bước nhảy vọt về điểm số: So với mạng LSTM tự xây dựng (ROUGE-1 chỉ đạt quanh mức 0.21), ViT5 đã chứng minh sự ưu việt của kiến trúc Transformer kết hợp với quá trình Pre-training (học trước). Mô hình không phải học lại từ đầu cấu trúc tiếng Việt mà chỉ cần dùng khối kiến thức khổng lồ có sẵn để tóm tắt văn bản.

- Đánh giá định tính: Khi đọc thử các bản tóm tắt sinh ra, câu văn rất tự nhiên, chuẩn ngữ pháp tiếng Việt và hoàn toàn không gặp hiện tượng lặp từ vô nghĩa như các mô hình cũ. Tỷ lệ nén lên tới hơn 92% chứng tỏ mô hình có khả năng chọn lọc thông tin cốt lõi (Gisting) xuất sắc thay vì chỉ cắt xén ngẫu nhiên.

- Ý nghĩa bản lề: Kết quả Inference ban đầu này khẳng định ViT5 là lựa chọn hoàn toàn đúng đắn. Nó tạo tiền đề cực kỳ vững chắc để sang Tuần 5-6, khi mô hình được tinh chỉnh (Fine-tuning) sâu hơn bằng hàng ngàn bài báo chuyên ngành y khoa, ViT5 chắc chắn sẽ nắm bắt chính xác các thuật ngữ y học phức tạp (tên bệnh, liều lượng) để đem lại kết quả hoàn thiện nhất.

3.2.2. Công tác thu thập dữ liệu (Data Crawling):

Dữ liệu là yếu tố cốt lõi trong học máy. Nhằm phục vụ cho quá trình tinh chỉnh (Fine-tuning) mô hình ở giai đoạn tiếp theo, nhóm đã tiến hành thu thập tự động các bài báo chuyên ngành Y khoa.

Kết quả: Thu thập thành công tập dữ liệu thô gồm các file .parquet - là các thông tin đa chủ đề để train lần đầu cho mô hình hiểu tiếng việt sau đó sẽ train lần 2 với các file csv - là các file chứa các bài báo về y khoa. Hiện tại sinh viên đã tìm được hơn 20000 bản ghi thô (bao gồm nội dung bài báo và đoạn tóm tắt tương ứng). Bộ dữ liệu này sẽ tiếp tục được tiền xử lý và làm sạch trong tuần tiếp theo.

3.3. Kết luận và Định hướng kỳ tiếp theo

Trong 4 tuần thực tập, sinh viên đã bám sát đề cương, hoàn thành xuất sắc việc xây dựng nền tảng từ cơ bản đến nâng cao: hoàn tất thuật toán TextRank (Thế hệ 1), mạng Seq2Seq LSTM (Thế hệ 2), thiết lập mã nguồn đánh giá Transformer ViT5 (Thế hệ 4) và hoàn thành bước thu thập dữ liệu y khoa.

Định hướng Tuần 5 - 6: Tiền xử lý dữ liệu Y khoa (Preprocessing), áp dụng thuật toán Text Chunking cho văn bản dài, và tiến hành Fine-tuning mô hình ViT5 trên tập dữ liệu này. Xây dựng giao diện ứng dụng web trực quan và hoàn thiện toàn bộ báo cáo thực tập tổng hợp.

TÀI LIỆU THAM KHẢO

- [1] R. Mihalcea and P. Tarau, "TextRank: Bringing Order into Texts," in *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, Barcelona, Spain, Jul. 2004, pp. 404–411. [Online]. Available: <https://aclanthology.org/W04-3252>. [Accessed: Jul. 11, 2026].
- [2] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," in *Advances in Neural Information Processing Systems*, vol. 27, 2014. [Online]. Available: <https://arxiv.org/abs/1409.3215>. [Accessed: Jul. 11, 2026].
- [3] D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," in *3rd International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, May 2015. [Online]. Available: <https://arxiv.org/abs/1409.0473>. [Accessed: Jul. 11, 2026].
- [4] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in *Text Summarization Branches Out*, Barcelona, Spain, Jul. 2004, pp. 74–81. [Online]. Available: <https://aclanthology.org/W04-1013>. [Accessed: Jul. 11, 2026].
- [5] "PyTorch Documentation - Torch.nn Modules," *PyTorch Foundation*. [Online]. Available: <https://pytorch.org/docs/stable/nn.html>. [Accessed: Jul. 11, 2026].